The $1/\mathcal{W}$ Law: An Analytical Study of Context-Length Routing Topology and GPU Generation Gains for LLM Inference Energy Efficiency

Huamin Chen[1] Xunzhuo Liu[1] Yuhan Liu[2]
Junchen Jiang[3] Bowei He[4*] Xue Liu[4] Chenxu Niu[5]

[1]vLLM Semantic Router Project
[2]University of Chicago [3]Tensormesh Inc / University of Chicago
[4]MBZUAI / McGill University [5]NVIDIA

2026

Abstract

How many tokens can a GPU inference cluster deliver per watt? Across deployments of identical hardware, the answer varies by 40×—not because of software inefficiency, but because of the serving context window. We derive the $1/\mathcal{W}$ *law*: tokens per watt halves every time the context window doubles. A larger context window shrinks the KV-cache concurrency limit while leaving GPU power draw roughly unchanged. At 64K context, an H100 holds 16 sequences in flight (tok/W = 1.5); at 4K context, the same H100 holds 256 sequences (tok/W = 17.6).

Routing topology—which determines the effective context window each GPU services—remains a stronger energy lever than buying newer hardware. Working from published H100 power measurements [Chung et al., 2026], a calibrated logistic power model [Liang et al., 2026], a roofline throughput model [Xu et al., 2026], and a low-concurrency B200 calibration from one 8×B200 vLLM serving run, we derive these results analytically using the inference-fleet-sim framework [Chen et al., 2026b]. Two-pool context-length routing (FleetOpt [Chen et al., 2026a]) still delivers roughly 2.5× over a homogeneous H100 fleet, while the calibrated B200 generation gain rises to roughly 2× at fixed topology. On the short-context-dominant Azure trace, combining FleetOpt with calibrated B200 reaches 5.76× over the H100 homogeneous baseline. H100 results are anchored to published measurements; B200 remains **FAIR quality** because the calibration uses only a single-concurrency serving point rather than a full power-vs-batch sweep.

For MoE models, active-parameter weight streaming adds a third lever. Qwen3-235B-A22B (22B active) reaches roughly 37.8 tok/W at 8K context on H100—5.1× better than Llama-3.1-70B—because decode time scales with activated weights, not total parameters. MoE dispatch overhead is excluded, so this is an upper bound.

1 Introduction

Generative AI workloads now consume a measurable fraction of data center power, and that fraction is growing. Unlike batch compute jobs, LLM inference has a nonlinear relationship between load and power: a GPU serving a single request idles near 43% of its thermal design power (TDP), while the same GPU running a large batch approaches 86% TDP [Chung et al., 2026]. The number of requests processed in parallel is therefore not just a throughput concern—it directly determines how much useful work you get per joule.

*Corresponding author: `Bowei.He@mbzuai.ac.ae`

The main thing controlling how many requests a GPU can hold in parallel is the serving context window. A KV cache of fixed VRAM can hold fewer concurrent sequences as the per-token memory footprint grows. A Llama-3.1-70B deployment on H100 (80 GB, TP=8 with tensor-parallel KV sharding) [Meta AI, 2024] can hold 256 concurrent 4K-token sequences but only 16 concurrent 64K-token sequences. Tokens per watt drops by the same factor across those two operating points—roughly 12×—and extends to a nearly 40× spread across the full 2K to 128K context range.

GPU vendors have been responding to growing context demands with more powerful accelerators. B200 replaces H100 with 2.4× the memory bandwidth and 2.4× the VRAM—but also a TDP that is 43% higher (1,000 W vs. 700 W). Whether you actually get better tokens per watt from a B200 depends entirely on the context window distribution your workload presents at inference time.

This paper works out the quantitative relationships between context window, routing topology, hardware generation, and fleet energy efficiency. We connect three existing tools—the logistic GPU power model of Liang et al. [Liang et al., 2026] calibrated to ML.ENERGY measurements [Chung et al., 2026], the AIConfigurator roofline [Xu et al., 2026], and the `inference-fleet-sim` planner [Chen et al., 2026b]—to derive the $1/\mathcal{W}$ law (Section 3.1): tok/W halves every time the serving context window doubles, because halving concurrency halves throughput while power stays flat. This makes the context window—not the GPU—the primary energy knob.

From there we show (Section 4.2) that topology and hardware generation remain approximately multiplicative levers after calibration. FleetOpt routing gives roughly 2.5× over homogeneous on H100 and 2.8× on calibrated B200 for Azure; upgrading from H100 to calibrated B200 gives roughly 2× at fixed topology. On short-context-dominant workloads the two effects combine to roughly 5.8×. Neither lever alone gets you close to that total. Section 3.2 examines MoE models, where sparse activation collapses the per-iteration weight-streaming time and adds a third, architecture-level efficiency dimension.

2 Background

2.1 GPU Power Model

Liang et al. [Liang et al., 2026], studying GPU power as a knob for grid voltage regulation, characterized H100 power under LLM inference as a logistic curve in b, the number of concurrently in-flight sequences (`max_num_seqs` in vLLM):

$$P(b) = \frac{P_{\text{range}}}{1 + e^{-k(\log_2 b - x_0)}} + P_{\text{idle}}, \tag{1}$$

where P_{idle} is idle power, $P_{\text{range}} = P_{\text{nom}} - P_{\text{idle}}$ is the dynamic range, k controls how steeply power rises with batch size, and x_0 is the half-saturation point. Chung et al. [Chung et al., 2026] provide direct measurements for H100-SXM5: $P_{\text{idle}} = 300$ W at b=1 and $P_{\text{nom}} = 600$ W at b=128, both within 3% fit error. The fitted logistic parameters are $k = 1.0$ and $x_0 = 4.2$, meaning power saturates around $2^{4.2} \approx 18$ concurrent sequences.

For B200-SXM, we now use a partial empirical calibration from one 8×B200 node running vLLM 0.19.1 with Llama-3.1-70B at 8K context and `max_num_seqs`= 1. The measured states are: true unloaded idle ≈ 191 W/GPU, model loaded but not serving ≈ 251 W/GPU, and active inference ≈ 513 W mean / 606 W p95 per GPU. Because the simulator assumes model weights are already resident, we map the loaded-idle state to $P_{\text{idle}} = 251$ W and the p95 active state to $P_{\text{nom}} = 606$ W. With $k = 1.0$, fitting Eq. (1) to the b=1 active mean gives $x_0 \approx -1.03$. Throughout the paper, H100 power estimates remain **HIGH quality** (directly measured), while B200 power estimates are **FAIR quality**: empirically anchored at one serving point, but not yet constrained by a full power-vs-concurrency sweep.

KV-cache storage assumption. Fleet results assume tensor-parallel sharding of KV heads: with TP=8 and Llama-3.1-70B's $n_{\text{kv}} = 8$ GQA heads, each GPU stores one KV head, giving $\kappa \approx 55$ KB/token. This matches the empirically calibrated H100 profile ($n_{\max} = 128$ at 8K context, implying 55 KB/token from the known VRAM budget) and is consistent with vLLM's default behavior for GQA models. B200 KV capacity is then scaled proportionally by the memory-budget ratio—156 GB usable vs. 60 GB on H100—giving a 2.62× larger KV budget and $n_{\max}^{\text{B200}} \approx 2.62 \times n_{\max}^{\text{H100}}$ at any context window.

2.2 Token/Watt: Definition and Decomposition

Single-GPU tok/W. For a GPU holding n_{active} concurrent sequences with mean KV context length $\bar{L}$, tokens per watt is:

$$\text{tok/W}_{\text{GPU}} = \frac{n_{\text{active}}/\tau(n_{\text{active}}, \bar{L})}{P(n_{\text{active}})}, \tag{2}$$

where $\tau(n, \bar{L}) = W + H(\bar{L}) \cdot n$ is the per-iteration decode latency from the roofline model. Here W is the weight-streaming time per iteration (proportional to model size and inversely proportional to memory bandwidth), and $H(\bar{L}) = H_0 \cdot \bar{L}/L_{\text{calib}}$ is the per-sequence KV-scan overhead, which grows linearly with context length.

The maximum concurrency $n_{\max}(\mathcal{W})$ for a context window $\mathcal{W}$ is set by the KV-cache memory budget:

$$n_{\max}(\mathcal{W}) = \left\lfloor \frac{V_{\text{KV}}}{\kappa \cdot \mathcal{W}} \right\rfloor, \tag{3}$$

where V_{KV} is the KV-cache VRAM after model weights and κ is the KV bytes per token. Since both $n_{\max}$ and $H(\bar{L})$ scale linearly with $\mathcal{W}$, throughput at full concurrency scales as $1/\mathcal{W}$ while power stays roughly flat. This is the origin of the $1/\mathcal{W}$ law.

Fleet-level tok/W. For a heterogeneous fleet of pools $i \in \{1, \ldots, K\}$, each with n_i GPU instances:

$$\text{tok/W}_{\text{fleet}} = \frac{\sum_i \lambda_i \cdot \overline{L}_{\text{out},i}}{\sum_i n_i \cdot P(n_{\text{act},i})}, \tag{4}$$

where λ_i is the request arrival rate to pool i, $\overline{L}_{\text{out},i}$ is the mean output length, and $n_{\text{act},i} = \rho_i \cdot n_{\max,i}$ is the mean in-flight batch at utilization ρ_i. Fleet tok/W does not reduce to a single GPU-level quantity because n_i and $P(n_{\text{act},i})$ vary across pools and must be weighted by GPU count.

3 Single-GPU Token Economy

3.1 Context Window as the Primary Lever

Table 1 shows how concurrency and tok/W change as the context window grows, for Llama-3.1-70B on H100-SXM5 and B200-SXM (TP=8, fp16). H100 uses the empirically calibrated profile; B200 values scale proportionally via the 2.62× KV-budget ratio from the roofline model [Xu et al., 2026, Chen et al., 2026b].

The pattern in the table is striking. Tok/W halves with every doubling of the context window, for both GPU generations. This is the $1/\mathcal{W}$ law, and it is not approximate: doubling $\mathcal{W}$ halves $n_{\max}$ (Eq. (3)), which halves throughput (since $\tau \propto n_{\max} \cdot H \propto \mathcal{W}$), while power stays nearly constant. The GPU burns the same watts whether it holds 512 sequences or 8; the difference is entirely how much work it delivers per joule.

B200 shifts the curve up but does not change the slope. Under the calibrated power model, B200 achieves roughly 1.5–2.5× higher tok/W than H100 at matching context windows, driven

Table 1: $n_{\max}$ and tok/W vs. context window for Llama-3.1-70B (TP=8, fp16). H100 power is directly measured (HIGH quality); B200 power uses a single-concurrency empirical anchor plus analytical scaling (FAIR quality).

	H100-SXM5			B200-SXM6		
Context (K)	$n_{\max}$	P_{sat}	tok/W	$n_{\max}$	P_{sat}	tok/W
2	512	598 W	35.0	1725	795 W	37.09
4	256	593 W	17.6	872	713 W	30.34
8	128	583 W	8.97	439	655 W	20.55
16	64	557 W	4.69	220	596 W	13.02
32	32	507 W	2.58	110	590 W	7.89
64	16	435 W	1.50	55	637 W	4.30
128	8	369 W	0.88	28	723 W	1.14

by a larger KV-cache budget (156 GB free vs. 60 GB on H100), faster weight streaming (W = 2.95 ms vs. 6.72 ms for 70B at TP=8), and correspondingly higher $n_{\max}$. But the halving-per-doubling relationship holds just as tightly on B200.

B200's advantage narrows at long context. At 64K, B200 is only 1.81× better than H100 (2.71 vs. 1.50 tok/W), down from 2.47× at 4K. The reason is no longer a large projected idle penalty; instead, the calibrated B200 serving curve is already near its active ceiling across these operating points, staying close to 606 W while H100 power falls more noticeably at low batch. At 64K, the B200 still has far more concurrency (41 sequences vs. 16), but the watt denominator no longer drops with context length the way it does on H100. We return to the fleet-level implications in Section 4.

3.2 Model Architecture Effects

Table 2 compares single-GPU tok/W at $n_{\max}$ across model families for an 8K context window.

Table 2: Single-GPU tok/W at $n_{\max}$ (8K context). Dense model values use ComputedProfile throughout (H100 cross-validated against ML.ENERGY v3.0, HIGH quality; B200 uses a single-concurrency measured power anchor, FAIR quality). Models: Llama-3.1 family [Meta AI, 2024]; Qwen3-235B-A22B [Qwen Team, 2025]; DeepSeek-V3 [DeepSeek-AI, 2025]. MoE rows (†) override W with active-parameter-only streaming time (W = active_param_bytes / mem_bw), which is a *lower bound* on true W (excludes MoE dispatch overhead). DeepSeek-V3 active params: ≈37B estimated (671B total, 256 experts, top-8).

		H100-SXM5			B200-SXM		
Model	TP	$n_{\max}$	tok/s	tok/W	$n_{\max}$	tok/s	tok/W
Llama-3.1-8B	1	58	3,350	6.46	148	9,962	16.43
Llama-3.1-70B	8	22	2,716	7.41	58	12,960	21.39
Llama-3.1-405B	8	1	26	0.09	17	1,009	1.67
Qwen3-235B-A22B†	8	24	11,521	37.82	146	80,584	132.93
DeepSeek-V3† (fp8)	8	1	646	2.14	11	8,162	13.55

Llama-3.1-405B is effectively unusable for long-context serving on H100 at 8K. With all 405B parameters loaded across eight GPUs in fp16, the KV budget is nearly exhausted by model weights alone, leaving room for only one sequence in flight. Tok/W is negligible because the GPU is spending most of its power just holding the model warm between sequential requests. B200's additional memory lifts $n_{\max}$ to 17, which is still modest but gives measurable efficiency. This model really needs hardware with large VRAM—B200, GB200, or a shorter context window.

The MoE numbers in the table warrant explanation. For Qwen3-235B-A22B, we override the default W calculation to use only the 22B active parameters rather than the full 235B model. In a dense model, every weight is touched every iteration, so W scales with total parameters. In a MoE model, only a small subset of experts is activated per token, so the weight-streaming time scales with the active count—roughly $22/235 \approx 9\%$ of a dense 235B model. This gives $W \approx 1.6$ ms on H100 (vs. 6.7 ms for the dense 70B baseline), which is why Qwen3-235B achieves dramatically higher tok/s and tok/W despite a much larger total parameter count.

The MoE values in the table are upper bounds, and the gap from reality can be large. Routing tokens to experts requires an all-to-all dispatch across TP ranks, which adds latency on top of active-weight streaming. Depending on network topology and expert load balance, dispatch can add anywhere from a few to tens of milliseconds per iteration. At 10 ms of dispatch overhead, the Qwen3 H100 advantage over Llama-70B shrinks from 5× to around 1.5×. The active-parameter estimate gives a principled lower bound on W; empirical measurements on real hardware would substantially narrow the range.

Looking at B200 across models, the generation multiplier is anything but uniform. Under the calibrated power model, for sub-100B dense models it ranges from 2.5× to 2.9×, which is consistent with the bandwidth and memory improvements. But for models previously stuck at $n_{\max} \approx 1$—the 405B case—the jump is dramatic: from 0.09 to 1.67 tok/W, an $\approx 19\times$ improvement. This happens not because B200 is fundamentally more efficient at that operating point, but because the added VRAM lets the GPU escape the near-idle serving regime where it was burning nearly P_{idle} watts to deliver essentially nothing.

4 Fleet Topology and Token Economy

4.1 Fleet-Level Analysis Setup

When serving a production workload at a given SLO, an operator provisions enough GPUs to sustain the request arrival rate. Fleet tok/W depends on both per-GPU efficiency and the number of GPUs required—a topology that sends all traffic to a 64K context pool forces every GPU to run at the low-efficiency end of the $1/\mathcal{W}$ curve.

All fleet results in this section use Llama-3.1-70B (TP=8, fp16), sized to meet P99 TTFT ≤ 500 ms at $\lambda = 1{,}000$ req/s. H100 uses the empirically calibrated profile with TP-sharded KV ($n_{\max} = 128$ at 8K context); B200 uses a profile scaled proportionally from H100 by the 2.62× KV-budget ratio (Section 2.1).

4.2 Topology Effects Across GPU Generations

Table 3 shows fleet tok/W for three topologies on H100 and B200 across two production workload traces.

The topology gain is consistent across both GPU generations. Define the topology improvement for GPU G as $\text{tok/W}_{\text{FleetOpt}}(G)/\text{tok/W}_{\text{Homo}}(G)$, and the generation improvement for topology T as $\text{tok/W}_{\text{B200}}(T)/\text{tok/W}_{\text{H100}}(T)$. For Azure:

$$\Delta_{\text{topo}}(\text{H100}) = 14.08/5.58 = 2.52, \quad \Delta_{\text{topo}}(\text{B200}) = 32.12/11.42 = 2.81.$$

$$\Delta_{\text{gen}}(\text{Homo}) = 11.42/5.58 = 2.05, \quad \Delta_{\text{gen}}(\text{FleetOpt}) = 32.12/14.08 = 2.28.$$

The topology gain stays in the same ballpark between H100 and calibrated B200 (2.52 vs. 2.81), and the generation gain stays near 2× between homogeneous and FleetOpt (2.05 vs. 2.28). The two levers still operate on largely different dimensions, though the calibrated B200 power floor makes the generation effect somewhat stronger for short-context-heavy topologies. For Azure, calibrated B200 FleetOpt delivers $32.12/5.58 = 5.76\times$ over the H100 homogeneous

Table 3: Fleet token efficiency at $\lambda = 1{,}000$ req/s. H100 profile: empirically calibrated (HIGH quality); B200 profile: scaled from H100 by the 2.62× KV-budget ratio with a single-concurrency measured power anchor (FAIR quality). Topology: Homo = homogeneous 64K fleet; Pool = two-pool context routing; FleetOpt = optimal γ^* from Chen et al. [2026a]. Workload traces: Azure = Azure LLM Inference Trace [Patel et al., 2024]; LMSYS = LMSYS-Chat-1M [Zheng et al., 2023].

Workload	B_{short}	Topology	GPU	GPUs	kW	tok/W	vs H100 Homo
Azure (Arch. I)	—	Homo 64K	H100	141	58.3	5.58	—
	4K	Pool routing	H100	68	32.0	9.16	+64%
	4K/γ^*=2	FleetOpt	H100	40	23.1	14.08	+152%
	—	Homo 64K	B200	47	28.5	11.42	+105%
	4K	Pool routing	B200	25	15.1	19.37	+247%
	4K/γ^*=2	FleetOpt	B200	17	10.1	32.12	**+476%**
LMSYS (Arch. I)	—	Homo 64K	H100	69	28.5	4.77	—
	1.5K	Pool routing	H100	38	16.4	7.91	+66%
	1.5K/γ^*=2	FleetOpt	H100	29	12.9	10.30	+116%
	—	Homo 64K	B200	24	14.5	9.36	+96%
	1.5K	Pool routing	B200	16	9.7	13.42	+181%
	1.5K/γ^*=2	FleetOpt	B200	12	7.3	18.28	**+283%**

baseline. Neither lever alone gets you close—FleetOpt with H100 gives 2.52× and calibrated B200 with a homogeneous fleet gives 2.05×.

In practice, topology work and hardware upgrades are complementary investments, not substitutes. Operators who only buy better GPUs without rethinking topology capture about 2× and still leave a large topology gain on the table. Operators who invest in two-pool routing on H100 capture 2.52× and then can stack another ≈ 2.3× on top with a calibrated B200 upgrade in the Azure setting—arriving at a fleet that delivers 5.76× better energy efficiency than the H100 homogeneous baseline.

5 Routing Topology Design Space

5.1 Context-Window Routing vs. Semantic Routing

The two-pool design in Section 4 partitions traffic by prompt length: requests shorter than B_{short} go to the short pool (configured for a small context window), and the rest go to the long pool. An alternative is semantic routing: send simple or short requests to a small model (e.g., Llama-3.1-8B) and complex or long requests to a large model (e.g., Llama-3.1-70B). Table 4 compares the per-pool efficiency of the two approaches at $\rho = 0.85$ utilization.

Table 4: Single-GPU tok/W for context-window routing vs. semantic routing (H100-SXM5, $\rho = 0.85$ utilization, 8K or 64K context windows).

Pool type	Model	Context	n_{active}	P (W)	tok/W
Context short (70B@8K)	Llama-3.1-70B	8K	109	578	8.77
Context long (70B@64K)	Llama-3.1-70B	64K	14	413	1.52
Semantic small (8B@8K)	Llama-3.1-8B	8K	49	506	6.24
Semantic large (70B@64K)	Llama-3.1-70B	64K	14	413	1.52

The long pool is the binding constraint in both schemes. At 64K context, any model saturates at $n_{\max} = 16$ sequences per TP=8 group and delivers 1.52 tok/W—less than one-

sixth of the short-pool value. When more than 89% of traffic is short-context (as in the Azure trace), the long pool is nearly idle yet still draws 413 W per group, dragging down the fleet average. The $1/\mathcal{W}$ law explains the magnitude: the 8× context ratio between short and long pools implies 8× fewer concurrent sequences, which translates to roughly 8× lower tok/W. No amount of smarter routing eliminates this gap—it is a physical consequence of KV memory limits.

The choice between context routing and semantic routing is an efficiency tie for the long pool (both land at 1.52 tok/W at 64K) and a near-tie for the short pool (8.77 vs. 6.24 tok/W on a per-group basis). The 70B context-short pool appears more efficient per group because TP-sharded GQA allows more concurrent sequences (109 vs. 49). But on a per-physical-GPU basis, the 8B semantic pool wins: a single 8B GPU handles as many sequences as eight 70B TP-sharded GPUs combined, at a fraction of the cost. The bottom line is that the energy case for semantic routing depends not on tok/W but on whether the 8B model can satisfy output quality requirements for the short-context fraction of traffic.

5.2 Quantization Effects

Quantization to fp8 or int4 cuts weight bytes by 2–4× [Micikevicius et al., 2022, Frantar et al., 2023], proportionally reducing W. For H100+70B, fp8 gives $W \approx 3.36$ ms (vs. 6.72 ms in fp16), which roughly doubles tok/W at any fixed concurrency. The benefit is largest for dense models currently bottlenecked by weight streaming at moderate concurrency, and smallest for MoE models where W is already small relative to KV overhead.

6 GPU Generation Comparison

Table 5 summarizes key hardware parameters and their impact on tok/W for Llama-3.1-70B (TP=8, fp16).

Table 5: GPU generation comparison for Llama-3.1-70B (TP=8, fp16) at 8K context. Power quality: H100 = HIGH (measured); B200 = FAIR (single-concurrency measured anchor); H200/GB200 = FAIR (projected, ±15%).

GPU	TDP (W)	P_{idle}	W (ms)	$n_{\max}$@8K	P_{sat} (W)	tok/W	\$/hr	tok/\$M
H100-SXM5	700	300	6.72	22	367	7.41	32.2	0.30M
H200-SXM	700	300	4.76	44	422	15.58	48.0	0.49M
B200-SXM	1000	251	2.95	58	606	21.39	64.0	0.73M
GB200-NVL	1200	516	2.95	65	755	18.49	80.0	0.63M

H200 is worth a closer look for Llama-3.1-70B at 8K context. It doubles $n_{\max}$ over H100 (44 vs. 22) via 4.8 TB/s HBM3e bandwidth (1.4× H100), cuts W from 6.72 ms to 4.76 ms, and carries the same TDP and idle power floor as H100. The result is a 2.1× tok/W improvement over H100 (15.58 vs. 7.41) at a rental cost roughly 50% higher.

B200 pushes further in absolute tok/W (21.39 vs. 15.58) by fitting still more sequences in flight ($n_{\max} = 58$ vs. 44). Under the calibrated serving model its effective idle floor is actually lower than the old TDP-fraction projection (251 W vs. the previous 430 W), while the 2.62× larger memory budget more than compensates for the higher TDP. On a tok/\$M basis, B200 comes out ahead: 0.73M vs. 0.49M tok/\$M/hr, because the throughput gain more than offsets the cost premium. At 8K context, B200 is the better choice on both absolute efficiency and cost efficiency.

GB200-NVL is a bit of a surprise. Despite having the same compute specs as B200-SXM, its per-GPU tok/W is lower (18.49 vs. 21.39) because the higher TDP (1,200 W per GPU equivalent) adds more power to the denominator than the slightly larger memory (200 GiB vs.

180 GiB) adds to the numerator. GB200's advantage shows up not in this configuration but for very large models like the 405B or DeepSeek-V3, where its memory lets $n_{\max}$ climb out of the near-zero regime.

7 Workload Archetype Effects

How much any topology helps depends entirely on the workload: the fraction of traffic at each context length determines how hard the long pool drags on the fleet average.

Short-context-dominant workloads (e.g., Azure). 89% of Azure Conversations requests fit within 4K tokens. The short pool handles nearly all traffic at high concurrency; the long pool is small and lightly loaded. FleetOpt on calibrated B200 reaches 32.1 tok/W—a +476% improvement over the H100 homogeneous baseline, composed of a 2.28× generation gain and a 2.81× topology gain that combine to 5.76×. This is the scenario where two-pool routing pays off most.

Dispersed workloads (e.g., agent-heavy). In agent-heavy traces, 74% of requests fit within 8K tokens, but the remaining 26% extend to 64K (p99 ≈ 32K). The long pool is large and dominates GPU-hours even with FleetOpt: a substantial fraction of traffic genuinely needs the long pool regardless of where you draw the split boundary. For these workloads, model architecture is the more productive energy lever—an MoE model with fast weight streaming benefits at every context length, not just the short tail.

Table 6 summarizes the recommended pairing of topology and GPU by archetype.

Table 6: Topology and GPU recommendations by workload archetype. Rankings are by tok/W; tok/$ may differ. B200/GB200 recommendations carry FAIR power model uncertainty; validate against real measurements before procurement.

Archetype	Traffic distribution	Best topology	Best GPU
Short-dominant (I)	>80% ≤8K tokens	FleetOpt two-pool	B200
Mixed (II)	50–80% ≤8K tokens	Pool routing	H200 or B200
Long-dominant (III)	<50% ≤8K tokens	Homo (long-pool only)	B200/GB200
MoE-capable models	Any distribution	Short pool + MoE	B200/GB200

8 Related Work

The GPU power model at the core of this analysis comes from Liang et al. [Liang et al., 2026], who showed that GPU power under LLM workloads follows a logistic curve in batch size and that `max_num_seqs` is the primary control knob for grid voltage regulation. ML.ENERGY Benchmark v3.0 [Chung et al., 2026] provides the measurement data used to calibrate our H100 profile, and the AIConfigurator [Xu et al., 2026] supplies the roofline decomposition of W and H. We extend these single-GPU results to the fleet level and show that topology and generation gains are approximately independent and multiplicative.

On the inference systems side, Splitwise [Patel et al., 2024] introduced prefill-decode disaggregation, and vLLM [Kwon et al., 2023] introduced PagedAttention for KV-cache management. Both optimize for latency and cost per token. FleetOpt [Chen et al., 2026a] and inference-fleet-sim [Chen et al., 2026b] approach the same problem from a fleet capacity perspective. The connection we make here is between routing topology and energy: topology controls which part of the $P(b)$ curve each GPU operates on, and therefore directly determines tok/W.

On the energy measurement side, Chung et al. [Chung et al., 2026] and TokenPowerBench [Niu et al., 2025] both confirm empirically that longer context degrades energy efficiency—TokenPowerBench reports roughly 3× higher energy per token when prompt length grows from 2K to 10K on H100. That direction is consistent with the $1/\mathcal{W}$ law, but the mechanism differs: TokenPowerBench captures the cost of processing more *input* tokens during prefill, while our law governs the decode throughput ceiling set by KV-cache concurrency at a fixed serving context window. Maliakel et al. [Maliakel et al., 2025] show that the decode phase dominates inference time (77–91%) and is largely memory-bandwidth-bound, directly supporting the mechanistic basis of the $1/\mathcal{W}$ law.

GreenServ [Ziller et al., 2026] is the closest contemporaneous work on routing for energy efficiency. It routes individual queries across a heterogeneous pool of 16 open-source LLMs using a multi-armed bandit policy, achieving a 31% energy reduction over random routing. The shared intuition is that not every request needs the most expensive GPU or model. Where GreenServ and this paper diverge is in the mechanism: GreenServ routes across model families to avoid oversized models for easy tasks, while we route by context window within a single model to exploit the $1/\mathcal{W}$ law. The structural guarantee from KV-cache capacity yields larger gains (roughly 2.5× at the fleet level) because context-window partitioning directly controls which segment of the logistic power curve each GPU occupies, rather than relying on learned query-difficulty estimates.

Routing topology as an explicit energy lever—distinct from model selection, quantization, or hardware choice—remains underexplored. Treating energy as a linear proxy for cost obscures the $1/\mathcal{W}$ relationship, which is multiplicative with hardware generation and independent of it.

9 Conclusion

Energy efficiency in LLM inference is shaped by three things that operators can actually control: GPU generation (which sets W, P_{idle}, and KV capacity), model architecture (which determines weight-streaming time and KV footprint), and routing topology (which determines the context window each GPU actually services). Of these, topology is the easiest to change today—you can restructure routing without buying anything new.

The $1/\mathcal{W}$ law explains why topology matters so much. Because tok/W halves every time the serving context window doubles, a routing strategy that confines short requests to a small-context pool effectively cuts the context window for the bulk of traffic. FleetOpt delivers roughly 2.5× over a homogeneous fleet, and calibrated B200 delivers roughly 2× over H100 at fixed topology. On the Azure trace these two improvements combine to 5.76×—$2.5 + 2.0 \neq 5.76$ because the gains stack on orthogonal dimensions and are only approximately constant across topologies.

MoE models have a natural tok/W edge from fast weight streaming—only the active experts get loaded per step—but putting a reliable number on that edge requires empirical MoE dispatch measurements that we do not have. The values in Table 2 are upper bounds.

All results are computed analytically via `inference-fleet-sim` [Chen et al., 2026b], with H100 power validated against ML.ENERGY v3.0 and B200 re-anchored to one low-concurrency serving calibration. H200 and GB200 remain projections. As fuller multi-concurrency measurement data for B200 and H200 becomes available, we expect the absolute numbers to shift further while the qualitative picture— topology first, generation second, architecture third—holds.

10 Limitations, Assumptions, and Future Work

10.1 Modeling Assumptions

A few simplifying assumptions limit the scope of the results.

TP-sharded GQA KV storage. Fleet results assume tensor-parallel sharding of KV heads, so each GPU stores n_{kv}/TP KV heads. For Llama-3.1-70B with $n_{\text{kv}} = 8$ and TP=8, this gives one head per GPU ($\kappa \approx$ 40–55 KB/token), maximizing $n_{\max}$. Models with fewer KV heads than TP ranks, or frameworks that replicate the full KV cache per GPU, will have lower $n_{\max}$ and correspondingly lower tok/W.

Continuous-batching decode. We model throughput as $n/\tau(n, \bar{L})$ with $\tau = W + H \cdot n$ from the roofline. This is accurate for continuous-batching decoders (vLLM, TRT-LLM) in steady state. It does not capture chunked prefill interleaved with decode, head-of-line blocking, or KV-cache eviction under memory pressure. All three effects reduce achievable throughput, so the analytical tok/W is an upper bound.

Steady-state traffic. Fleet sizing uses a steady-state queuing model at a fixed arrival rate, with context lengths drawn from a measured CDF. Real traffic is bursty and diurnal. Bursts require spare capacity beyond the steady-state minimum, which reduces average utilization and therefore tok/W. The reported values are attainable only under smooth, sustained load.

B200 power model quality. B200 power parameters are no longer pure TDP-fraction projections: they are anchored to one 8×B200 serving run at b=1, then extrapolated with the same logistic functional form used for H100. This substantially improves the low-concurrency estimate, but it is still not a full $P(b)$ calibration. We therefore keep a FAIR-quality label and expect the high-batch tail to remain uncertain until a proper concurrency sweep is measured.

Fixed TP=8, single model per pool. All experiments use TP=8 instances of the same model in each pool. Smaller TP configurations are more power-efficient per GPU at low concurrency but reduce per-GPU throughput. Mixed-TP or mixed-model pools are not explored.

Output-only energy accounting. Tok/W counts output tokens only. Prefill computation—which can account for 2–5× the power per request for long-prompt workloads—is excluded. For workloads with prompt-to-output ratios much greater than one (e.g., document summarization, RAG), the reported tok/W overestimates true energy efficiency.

10.2 Limitations

Only partial empirical validation on B200; none on H200. The B200 results now include a direct low-concurrency serving anchor, but we still have not measured the full vLLM power-vs-concurrency curve on B200, and we have no corresponding serving measurements on H200. We therefore cannot confirm that the calibrated B200 logistic shape remains correct at higher batch sizes, nor can we validate the H200 projection. Researchers with access to these platforms should measure and publish the full $P(b)$ curve to fill this gap.

Two-pool topology only. FleetOpt defines a single split boundary with one overflow parameter. Real fleets may benefit from three or more pools partitioned by context length, or from content-adaptive pools that split on semantic class. The multiplicative gain structure suggests that finer-grained topologies could compound further efficiency improvements, but this is not analyzed here.

Static fleet sizing. The analysis assumes a fixed, provisioned fleet. It does not model autoscaling, spot instances, or GPU sharing. In practice, dynamic scaling changes effective utilization and would alter the quantitative tok/W values.

Single workload CDF. Each experiment uses one of two traces. Real deployments serve mixed traffic with time-varying CDFs. The optimal split boundary B_{short} and overflow γ^* would need to be re-estimated as the workload shifts.

10.3 Future Work

Empirical B200/H200 power calibration. The most immediate follow-on is extending the current B200 b=1 anchor to a full logistic $P(b)$ sweep and collecting the same data for H200-SXM using the ML.ENERGY methodology [Chung et al., 2026]. That would upgrade the B200 results from FAIR to HIGH quality and replace the remaining H200 projection.

Prefill-decode disaggregation. Splitwise-style separation [Patel et al., 2024] assigns prefill and decode to different GPU pools. Combined with context-length routing, this could remove prefill energy from the output tok/W accounting and unlock further efficiency. Extending the power model to mixed prefill-decode batches is a natural next step.

Multi-pool topology optimization. Extending FleetOpt from two pools to $K \geq 3$, potentially with different context windows and GPU types, could be formulated as a mixed-integer program. The multiplicative independence result suggests each additional split could compound the topology gain further.

Carbon-aware joint optimization. Tok/W does not capture grid carbon intensity or electricity price. Minimizing \$/token or gCO_2/token requires accounting for PUE, time-of-day pricing, and grid mix. The per-GPU power model in this paper provides a natural starting point for a joint energy-cost objective.

Speculative decoding interaction. Speculative decoding changes the batch-size distribution in the decode loop, potentially increasing average n_{act}. Whether this improves or degrades tok/W depends on the draft model's power footprint and the verification hit rate—an open problem within the $P(b)$ framework.

Adaptive topology control. The split boundary is currently fixed offline from a historical CDF. An online controller that monitors the live request-length distribution and adjusts pool boundaries dynamically could maintain near-optimal tok/W under distribution shift.

References

Huamin Chen, Xunzhuo Liu, Yuhan Liu, Junchen Jiang, Bowei He, and Xue Liu. FleetOpt: Analytical fleet provisioning for LLM inference with compress-and-route as implementation mechanism. Preprint; arXiv ID forthcoming, 2026a.

Huamin Chen, Xunzhuo Liu, Yuhan Liu, Junchen Jiang, Bowei He, and Xue Liu. inference-fleet-sim: A queueing-theory-grounded fleet capacity planner for LLM inference. Preprint; arXiv ID forthcoming, 2026b.

Jae-Won Chung, Ruofan Wu, Jeff J. Ma, and Mosharaf Chowdhury. Where do the Joules go? Diagnosing inference energy consumption. arXiv preprint arXiv:2601.22076, 2026.

DeepSeek-AI. DeepSeek-V3 technical report. arXiv preprint arXiv:2412.19437, 2025.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

Zhirui Liang, Jae-Won Chung, Mosharaf Chowdhury, Jiasi Chen, and Vladimir Dvorkin. GPU-to-Grid: Voltage regulation via GPU utilization control. arXiv preprint arXiv:2602.05116, 2026.

Joe Maliakel, Reetuparna Das, David Blaauw, and Hun-Seok Kim. Characterizing energy-performance tradeoffs in LLM inference systems. arXiv preprint arXiv:2501.08219, 2025.

Meta AI. The Llama 3 herd of models. arXiv preprint arXiv:2407.21783, 2024.

Paulius Micikevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, Naveen Mellempudi, Stuart Oberman, Mohammad Shoeybi, Michael Siu, and Hao Wu. FP8 formats for deep learning. arXiv preprint arXiv:2209.05433, 2022.

Pengyuan Niu, Shu Shi, Yunzhe Li, Dake Chen, Junda Lin, Eric He, and Ren Wang. Token-PowerBench: A framework for benchmarking LLM inference power consumption at the token level. arXiv preprint arXiv:2512.03024, 2025.

Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024.

Qwen Team. Qwen3 technical report. arXiv preprint arXiv:2505.09388, 2025.

Tianhao Xu, Yiming Liu, Xianglong Lu, Yijia Zhao, Xuting Zhou, Aichen Feng, Yiyi Chen, Yi Shen, Qin Zhou, Xumeng Chen, Ilya Sherstyuk, Haorui Li, Rishi Thakkar, Ben Hamm, Yuanzhe Li, Xue Huang, Wenpeng Wu, Anish Shanbhag, Harry Kim, Chuan Chen, and Junjie Lai. AIConfigurator: Lightning-Fast configuration optimization for multi-framework LLM serving. arXiv preprint arXiv:2601.06288, 2026.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric P. Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. LMSYS-Chat-1M: A large-scale real-world LLM conversation dataset. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

Alexander Ziller, Konstantin Lange, Dmitry Morozov, Ramin Khalili, Kilian Pfeiffer, and Jörg Henkel. GreenServ: Plug-and-play energy-efficient LLM serving. arXiv preprint arXiv:2601.17551, 2026.

A Power Model Parameters

Table 7 summarizes power model parameters for each GPU. H100-SXM5 parameters are fitted to ML.ENERGY data [Chung et al., 2026] Figure 2 data (H100-SXM5, vLLM, Llama-3.1-class, $b \in \{1, 2, 4, 8, 16, 32, 64, 128, 256\}$; fit error $<3\%$). B200-SXM uses a single-concurrency empirical anchor from one 8×B200 serving run. H200 and GB200 remain first-principles projections.

Table 7: GPU power model parameters. HIGH = directly measured; FAIR = either a single-concurrency empirical anchor (B200) or a TDP-fraction projection calibrated to H100 ML.ENERGY data [Chung et al., 2026].

GPU	TDP (W)	P_{idle} (W)	P_{nom} (W)	k	x_0	Source	Quality
H100-SXM5	700	300	600	1.0	4.2	G2G Fig. 2 + ML.ENERGY	HIGH
H200-SXM	700	300	600	1.0	$5.5^{\dagger}$	TDP fraction	FAIR
B200-SXM	1000	251	606	1.0	-1.03	8×B200 vLLM b=1 anchor	FAIR
GB200-NVL	1200	516	1032	1.0	$6.8^{\dagger}$	TDP fraction	FAIR

$^{\dagger}x_0$ is derived from the roofline W/H ratio at the half-saturation batch size: $x_0 = \log_2(W/H_0)$, where H_0 is the KV overhead at the calibration context.

B Fleet Analyzer Code

All fleet tok/W results in this paper use the `fleet_tpw_analysis` API from `inference-fleet-sim` [Chen et al., 2026b]:

```
from fleet_sim.optimizer.base import fleet_tpw_analysis
result = fleet_tpw_analysis(
    pools=[
        dict(gpu=H100_80GB, cdf=short_cdf, lam=alpha*lam,
             max_ctx=B_short, label="short"),
        dict(gpu=H100_80GB, cdf=long_cdf, lam=(1-alpha)*lam,
             max_ctx=65536, label="long"),
    ],
    lam_total=lam, t_slo_ms=500.0,
)
print(result.fleet_tpw)     # tokens per joule
print(result.fleet_power_kw)
```

The API accepts any object satisfying the `GpuProfile` protocol (`ManualProfile` or `ComputedProfile`), which is what makes it straightforward to compare the measured H100 profile against B200 or GB200 projections on equal footing.